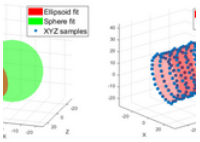


Object-oriented tools to fit/plot conics and quadrics

Version 1.2.11 (495 KB) by Matt J

A tool set for fitting and/or plotting various conics and quadric surfaces, e.g., ellipses, cylinders, spheres, planes, cones, and lines.

Follow

★★★★★ (7)
1.6K Downloads
Updated 8 Feb 2024
View License

Share | Open in MATLAB Online | Download

Overview | Functions | Examples | Version History | Reviews (7) | Discussions (16)

This FEX submission offers a tool set for fitting and plotting 2D conics (ellipses, circles, lines,...) as well as 3D quadric surfaces (ellipsoids, spheres, planes, cylinders, cones,...). Each type of fit is represented by an object within a class hierarchy. For each fit type, overloaded methods are provided for fitting the data, as well as post-plotting an overlay of the fit onto the raw data in a style similar to the Curve Fitting Toolbox. There are also methods for re-sampling the fit at a user-selected set of points.

Finally, the tools allow one to construct "ground truth" objects representing a conic/quadric with pre-selected parameters. This makes it possible to simply plot a desired ellipse or other supported shape regardless of whether it arises from a fitting process at all.

Currently, not all curve/surface types in the conic/quadric family are covered by this tool set, although I may add more over time. For now, I provide a subset of the ones that seem to be the most commonly encountered. Similarly, most fitting algorithms used by the tool set are very basic algebraic methods, although I may add more refined algorithms as the need and interest arise.

Various example uses of these tools are illustrated in the Examples tab.

Cite As

Matt J (2024). Object-oriented tools to fit/plot conics and quadrics (https://www.mathworks.com/matlabcentral/fileexchange/87584-object-oriented-tools-to-fit-plot-conics-and-quadrics), MATLAB Central File Exchange. Retrieved April 19, 2024.

MATLAB Release Compatibility

Created with R2020a
Compatible with R2018a and later releases

Platform Compatibility

☑ Windows ☑ macOS ☑ Linux

Tags

Add Tags

circle | cone | cone fit | conic | cylinder | ellipse | ellipsoid | fit | line | plane | quadric | sphere | surface

Others Also Downloaded

fit_ellipse
171 Downloads ★★★★★

Plane fit
23 Downloads ★★★★★

Ellipsoid fit
39 Downloads ★★★★★

circularFit

conicFit

cylindricalFit

ellipsoidalFit

ellipticalFit

linear2dFit

linear3dFit

planarFit

quadricFit

rightcircularconeFit

sphericalFit

Examples.mlx

Object-Oriented Tools for Fitting Conics

Table of Contents

Introduction

This FEX submission offers a tool set for fitting 2D conics (ellipses, circles, lines,...) as well as cylinders, cones,...). Each type of fit is represented by an object within a class hierarchy. For generating noisy test data, fitting the data, and visualizing and post-sampling the results.

Currently, not all curve/surface types in the conic/quadric family are covered by this tool set, a provide a subset of the ones that seem to be the most commonly encountered. Similarly, mos basic algebraic methods, although I may add more refined algorithms as the need and interes

Various example uses of these tools are illustrated in the sections below.

Two Dimensional Fitting

Fitting an Ellipse

First, we will synthesize noisy samples xy of a partial ellipse. To do so, we take advantage of `ellipticalFit` class. This method allows us to generate data for the ellipse, specifying its g minor axis radii `[a,b]`, and its clockwise rotation angle measured in degrees. Additive Gauss simulate measurement errors.

```
clear

[x0,y0]=deal(5,2);  %Center coordinates
[a,b]=deal(20,10);  %Major and minor axis radii
angle=30;           %Rotation angle in degrees

thetaSamps=[-70:5:50,100:5:150]; %Sample angles in degrees
sig=0.3;                         %Error standard deviation

xy=ellipticalFit.xysim([x0,y0],[a,b],angle, thetaSamps, sig); %Generate n
```

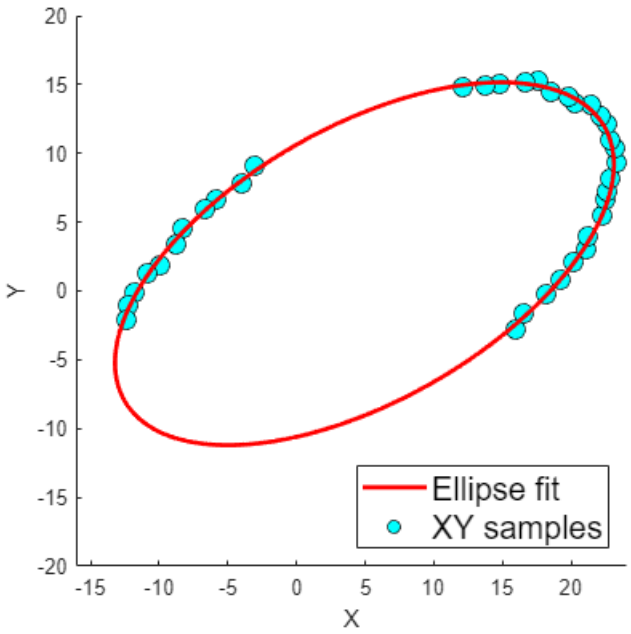
The sample locations `thetaSamps` correspond to initial azimuthal angles on the unit circle. Th achieve the final position of the ellipse, and the sample locations are transformed accordingly

Next, we will perform a fit to this data, returning the result as an `ellipticalFit` object called

```
fobj=ellipticalFit(xy), %Perform the fit
```

```
fobj =
ellipticalFit with properties: center: [4.9419 1.9780] a: 20.1074 b: 9.9625 angle: 29
```

```
[hFit,hData]=plot(fobj,{'LineWidth',2},{'MarkerFaceColor','c','MarkerEdge

legend([hFit,hData],'Ellipse fit','XY samples',...
                    'Location','southeast','FontSize',15);

hData.SizeData=80;
axis([-16 24 -20 20])
```



For visual verification, the `plot()` method of the `ellipticalFit` class overlays an implicit fu
the data samples, returning handles `hFit` and `hData` to each respectively. The `plot()` metho
of Name/Value pair arguments to `fimplicit()` and to `scatter()` to control the appearance
also be used to help generate a plot legend, as well as to make any desired post-adjustments

Fitting a Circle

Similar to the example above, we now generate noisy samples of a circular arc, specifying gr
circle.

```
clear

[x0,y0]=deal(12,10); %Center coordinates
radius=18;           %Circle radius

thetaSamps=linspace(0,120,10); %Sample angles in degrees
sig=0.6;                       %Error standard deviation

xy=circularFit.xysim([x0,y0], radius, thetaSamps, sig); %Generate noisy s
```

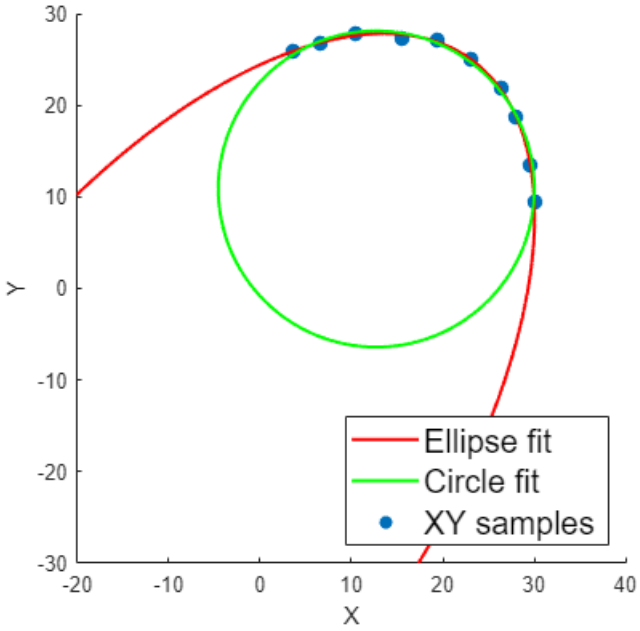
Because a circle is a special case of an ellipse, it is interesting to see the impact of parametri
ellipse. For data spanning a short arc, we can see how the lower degrees of freedom in the ci
more accurate estimate of the data's true underlying geometry.

```
fobjEllipse = ellipticalFit(xy);
fobjCirc    = circularFit(xy),
```

```
fobjCirc =
circularFit with properties: center: [12.7059 10.8636] radius: 17.2573
```

```
[hE,hD]=plot(fobjEllipse,{'Color','r'}); %Visualize ellipse fit

hold on
```

```
 hC=showfit(fobjCirc,'Color','g'); %Add the fitted circle to the plot
hold off

legend([hE,hC,hD],'Ellipse fit','Circle fit', 'XY samples',...
                  'Location','southeast','FontSize',15);

axis([-20 40 -30 30])
```



The usage of the `plot()` method is the same for circle fit objects as for ellipse fit objects. This and the `showfit()` methods of the fit objects can be used in concert to overlay multiple fits.

Fitting a 2D Line Segment

Here, we will fit noisy samples of a line segment, generated as follows. Sample locations are `t(i))*p1 + t(i)*p2` of two fixed points `p1` and `p2`.

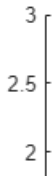
```
clear

p1=[-1,0];  p2=[1,+1.5];  %Simulated line segment end points

t=linspace(0,1,8); % Sample locations along line: t=0 at p1 , t=1 at p2.
sig=0.1;           % Error standard deviation

xy=linear2dFit.xysim(p1, p2, t, sig); %Generate noisy samples
```

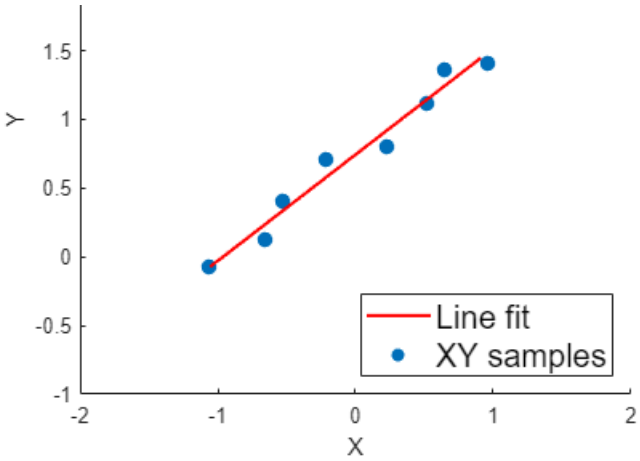
Similar to earlier examples, we generate a fit object and use its plot method to visualize the fit

```
fobjLine=linear2dFit(xy),
```

```
fobjLine =
linear2dFit with properties: p1: [-1.0645 -0.0768] p2: [0.9221 1.4552]
```

```
[hL,hD]=plot(fobjLine); %Visualize ellipse fit


legend([hL,hD],'Line fit', 'XY samples', 'Location','southeast','FontSize
axis([-2,2,-1,+3])
```





Three Dimensional Fitting

Fitting an Ellipsoid

In analogy with 2D ellipse fitting, the tool set also provides routines for fitting 3D ellipsoids. Th
those for the 2D case. In addition to providing ground truth center coordinates, the different a
order. The orientation of the ellipsoid is specified with a set of yaw, pitch, and roll angles `[y,p`
pose, the yaw, pitch, and roll are applied to an initial, unrotated version of the ellipsoid in whic
`[x,y,z]` axes respectively.

```
clear

[x0,y0,z0]=deal(5,2,7);  %Center coordinates
[a,b0,c]=deal(20,10,5);   %Axis radii
[y,p,r]=deal(40,-30,50); %Yaw, pitch, and roll angles specifying  ellipso

azimuthalSamps=0:10:270;  %Azimuthal angles of samples
elevationSamps=-40:10:40; %Elevation angles of samples
sig=0.3;                  %Error standard deviation

xyz=ellipsoidalFit.xyzsim([x0,y0,z0],[a,b0,c],[y,p,r],...
                          azimuthalSamps, elevationSamps, sig); %Generate
```

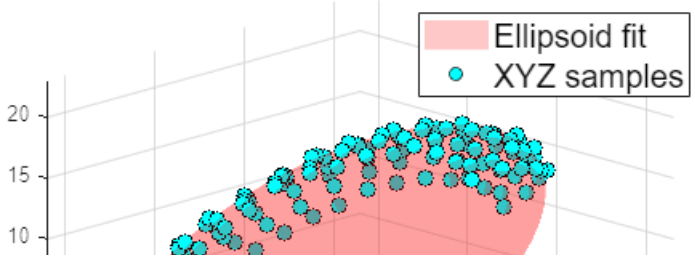
In the above, the vectors `azimuthalSamps` and `elevationSamps` are used to specify a curvili
the ellipsoid. These specify initial azimuthal and elevation coordinates of samples on the unit
translation are applied to bring the samples to their final positions.

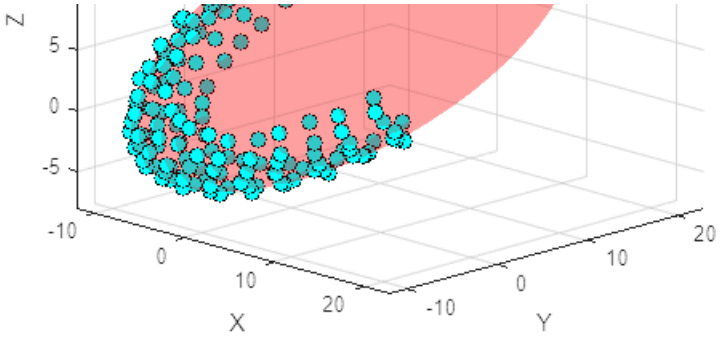
Performing and plotting the fit follows the same syntax as before.

```
fobjEllipsoid=ellipsoidalFit(xyz), %Perform the fit
```

```
fobjEllipsoid =
ellipsoidalFit with properties: center: [4.8881 1.9526 6.9725] a: 19.9838 b: 10.0250
49.7882
```

```
[hFit,hData]=plot(fobjEllipsoid,{'FaceAlpha',0.2},{'MarkerFaceColor','c',

legend([hFit,hData],'Ellipsoid fit','XYZ samples',...
                    'Location','northeast','FontSize',15);

axis([-12,+23,-12,+23,-8,+23]); view([45,16])
```



Fitting a Sphere

Simulating samples of a sphere proceeds much the same as for an ellipsoid, except fewer ge

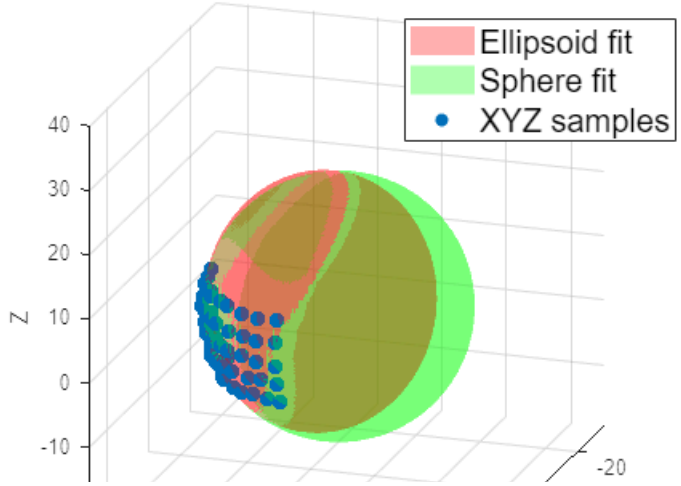
```
clear

[x0,y0,z0]=deal(5,2,7);  %Center coordinates
radius=20;               %Sphere radius

azimuthalSamps=linspace(0,80,10);  %Azimuthal angles of samples
elevationSamps=(-30:10:10);        %Elevation angles of samples
sig=0.3;                           %Error standard deviation

xyz=sphericalFit.xyzsim([x0,y0,z0],radius,...
                        azimuthalSamps, elevationSamps, sig); %Generate
```

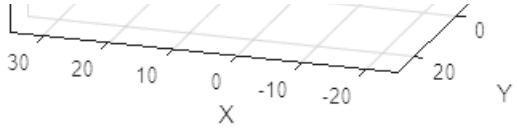
Below we compare an ellipsoid fit to a sphere fit in analogy with our earlier 2D example of circ

```
fobjEllipsoid=ellipsoidalFit(xyz); %Perform ellipse and sphere fits
fobjSphere=sphericalFit(xyz),
```

```
fobjSphere =
sphericalFit with properties: center: [4.8865 1.9498 6.7817] radius: 20.0674
```

```
[hE,hData]=plot(fobjEllipsoid); %Visualize the fit

hold on
 hS=showfit(fobjSphere,'FaceAlpha',0.3,'FaceColor','g');
hold off

legend([hE,hS,hData],'Ellipsoid fit','Sphere fit','XYZ samples',...
                     'Location','northeast','FontSize',15);

view([-161.9 18.6])
xlim([-25.0 35.0])
ylim([-32.0 28.0])
zlim([-19.7 40.3])
```





Fitting a Circular Cylinder

Generating noisy samples for a circular cylinder is similar to the case of an ellipsoid. Due to c
angles are needed to specify the cylinder's orientation. Also the lattice coordinates of the surf
coordinates, as opposed to spherical.

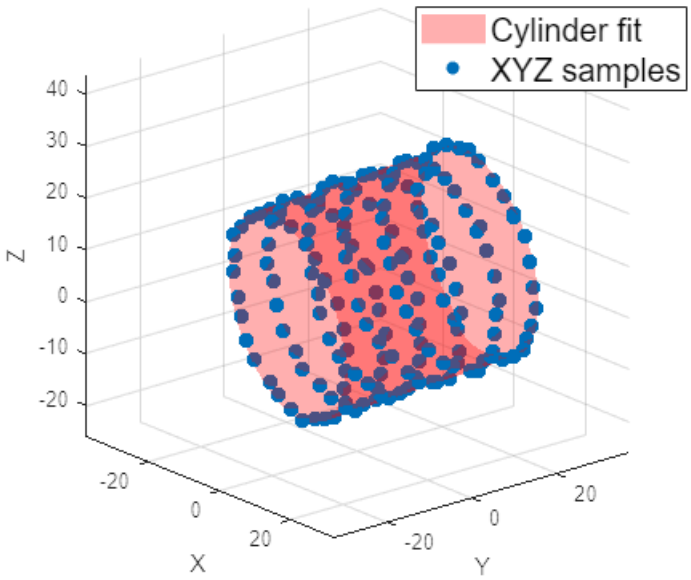
```
clear

[x0,y0,z0]=deal(5,2,7);  %Center coordinates
radius=20;               %Axis radii
[y,p]=deal(30,-25);      %Yaw and pitch angles (in degrees)

azimuthalSamps=linspace(0,360,30);      %Azimuthal angles of samples
axialSamps=linspace(-radius,radius,6);  %Samples length-wise along the cy
sig=0.6;                                %Error standard deviation

xyz=cylindricalFit.xyzsim([x0,y0,z0],radius,[y,p],...
                          azimuthalSamps,axialSamps, sig); %Generate nois
```

Unlike other fit types, the default cylinder fitting routine is semi-iterative. To estimate the 3D c
on `fminsearch` is used,

```
fobjCyl=cylindricalFit(xyz), %Perform cylinder fit
```

```
fobjCyl =
cylindricalFit with properties: center: [5.0049 1.9312 6.9554] radius: 19.9862 height
```

```
[hFit,hData]=plot(fobjCyl); %Visualize the fit


legend([hFit,hData],  'Cylinder fit','XYZ samples',...
                      'Location','northeast','FontSize',15);

xlim([-37.5 32.5]); ylim([-32.9 37.1]); zlim([-26.1 43.9]); view([50.0 20
```



Important: the height of the cylinder is ill-defined, since one cannot know what portion of its p

samples. Likewise, the position of its center is well-defined only up to a translation along the c
between the extreme-most samples along the estimated axis and the height is set as the axia

Fitting a Right Circular Cone

Simulating the surface fit of a right circular cone follows a very similar procedure to that of a c
specifying their ground truth azimuthal and axial locations:

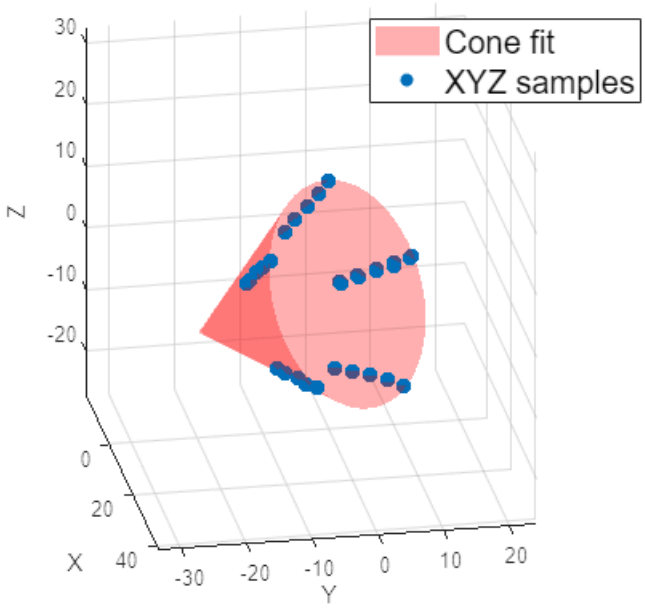
```
clear

[x0,y0,z0]=deal(0,-20,-10);  %Cone vertex coordinates
cone_angle=30;               %Angle at the vertex from axis to surface (i
[y,p]=deal(45,-25);          %Yaw and pitch angles (in degrees)

azimuthalSamps=linspace(0,360,6);   %Azimuthal angles of samples
axialSamps=linspace(20,30,5);       %Samples length-wise along the cone
sig=0.2;                            %Error standard deviation

xyz=rightcircularconeFit.xyzsim([x0,y0,z0],cone_angle,[y,p],...
                                azimuthalSamps,axialSamps, sig); %Generat
```

Also, as was the case for cylinders, the fitting of a cone uses a semi-iterative procedure base

```
fobjCone=rightcircularconeFit(xyz), %Perform cylinder fit
```

```
fobjCone =
rightcircularconeFit with properties: vertex: [-0.0560 -19.8045 -10.0358] cone_angle:
-25.0653
```

```
[hFit,hData]=plot(fobjCone); %Visualize the fit


legend([hFit,hData],  'Cone fit','XYZ samples',...
                      'Location','northeast','FontSize',15);

xlim([-19.5 41.8]); ylim([-33.4 24.2]); zlim([-27.2 32.6])
view([79.7 22.7])
```



Important: Similar to the case of a cylinder, the fitted height of the cone is ill-defined, since on
length is covered by the `xyz` samples. The code simply sets the height as the axial distance fi

Fitting a Plane

For plane fitting, the data simulation method `planarFit.xyzsim()` generates samples on a l
sample locations are of the form $h0+t1(i)*h1+t2(i)*h2$, where $h0$, $h1$, and $h2$ are fixed

sample locations are of the form b0+t1(i)*b1 +t2(j)*b2 where b0, b1, and b2 are fixed

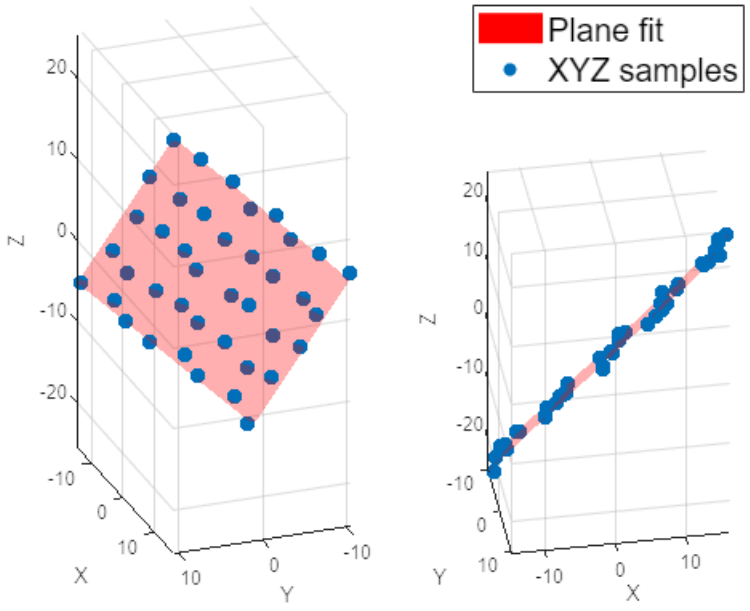
```
b0=[1,0,0]; %Plane reference point
b1=[0;1;1]; %Plane basis vector
b2=[1;0;1]; %Plane basis vector

t1=linspace(-10,10,8); % Grid coordinates along basis1
t2=linspace(-15,15,5); % Grid coordinates along basis2
sig=0.8;               % Error standard deviation

xyz=planarFit.xyzsim(b0,b1,b2,t1,t2,sig); %Samples are at b0+t1(i)*b1 +t2
```

The fit reports the normal vector to the plane of the samples, as well as the distance d of the p
normal.

```
fobjPlane=planarFit(xyz), %Perform plane fit
```

```
fobjPlane =
planarFit with properties: normal: [-0.5830 -0.5714 0.5775] distance: -0.6083
```

```
for i=1:2
    subplot(1,2,i);
    [hL,hD]=plot(fobjPlane); %Visualize the fit
end

legend([hL,hD],'Plane fit', 'XYZ samples', 'Location','northoutside','Fon
subplot(1,2,1)
view([-69.5 -24.6])
subplot(1,2,2)
view([9.6 -35.2])
```



Fitting a 3D Line Segment

Although a 3D line is not quadric surface, strictly speaking, the tool set handles this case for t
because it is a common case of interest. As in the 2D case, samples are generated by taking
p2.

```
clear;

p1=[-1,0,0];  p2=[1,+1.5,2];  %Simulated line segment end points

t=linspace(0,1,8); % Sample locations along line: t=0 at p1 , t=1 at p2.
sig=0.1;           % Error standard deviation

xyz=linear3dFit.xyzsim(p1, p2, t, sig); %Generate noisy samples
```

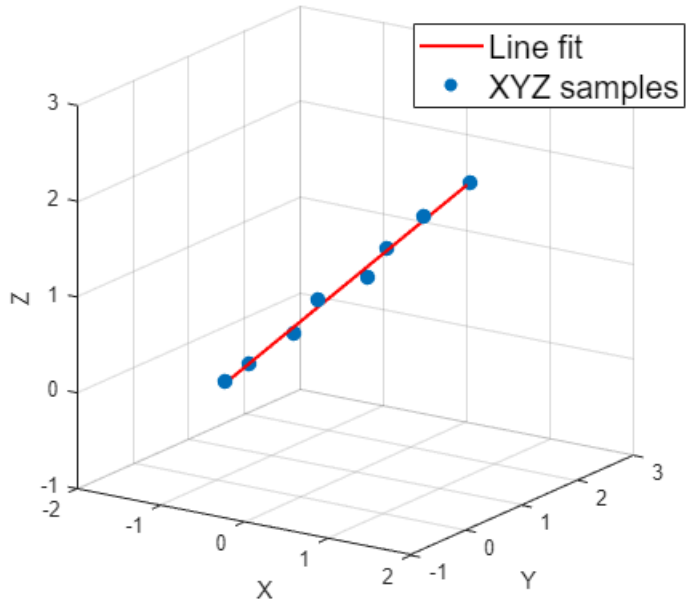
```
xyz=linear3dFit.xyzsim(p1, p2, t, sig); %Generate noisy samples
```

The syntax for fitting is analogous to other examples:

```
fobjLine=linear3dFit(xyz),
```

```
fobjLine =
linear3dFit with properties: p1: [-0.9155 0.0591 0.0252] p2: [1.0010 1.5302 2.0473]
```

```
figure;
[hL,hD]=plot(fobjLine,{'LineWidth',1.5,'Color','r'},{'SizeData',50}); %Vi

legend([hL,hD],'Line fit', 'XYZ samples', 'Location','northeast','FontSiz

axis([-2,2,-1,+3,-1,+3]); view([33.77 17.40])
```



Post-Fit Processing: Sampling the Fitted Curve or Surface

The various fit objects in the class hierarchy have a `sample()` method that can be used to ob
a fit. The help documentation gives the input syntax for these methods for each class of fit. Ty
same way as with the `xysim()` and `xyzsim()` static methods illustrated earlier. Below are se

Post-Sampling an Ellipse Fit

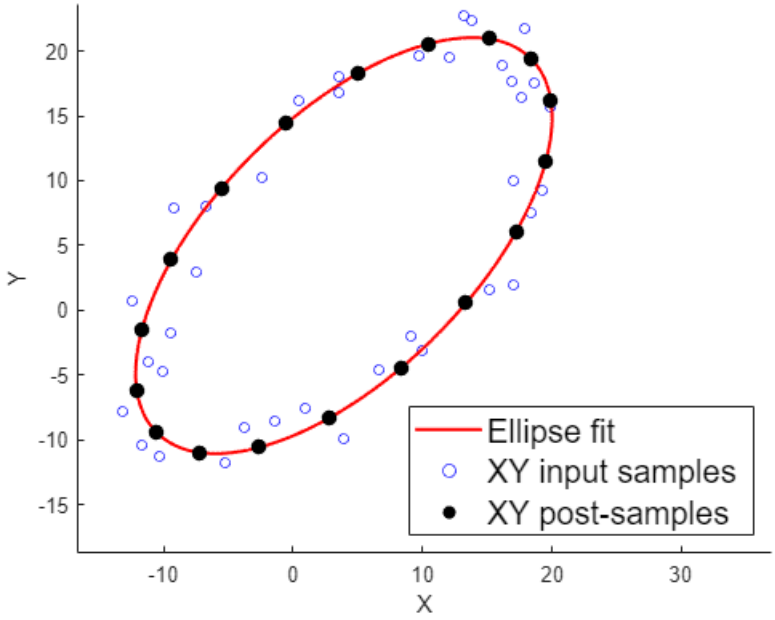
Here, we fit relatively noisy samples of an ellipse,

```
clear

xyInput=ellipticalFit.xysim([4,5],[20,10],45, (0:10:360), 2); %Simulate n

fobj=ellipticalFit(xyInput); %Perform the fit

[hFit,hData]=plot(fobj,{},{'MarkerFaceColor','none','MarkerEdgeColor','b'

xlim([-16.7 37.0])
ylim([-18.7 23.6])
```

and then post-sample more regularly around the ellipse.

```
xyPost=fobj.sample(0:20:360); %post-samples

hold on
hPost=scatter(xyPost{:},'filled','k','SizeData',50);
hold off

legend([hFit,hData,hPost],'Ellipse fit','XY input samples','XY post-sampl
```

```
                'Location','southeast','FontSize',15);
```



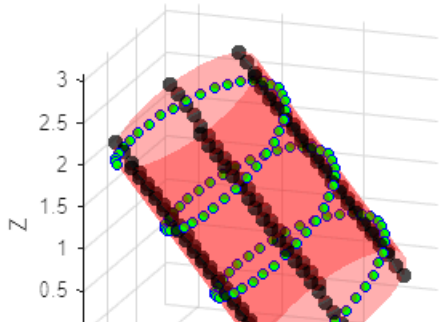
Post-Sampling a Cylinder Fit

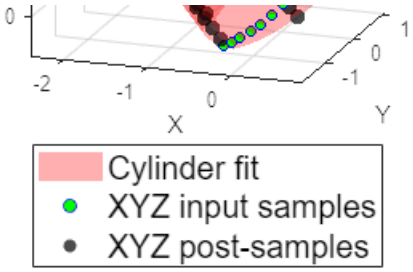
Here, we fit a cylinder to samples of a helical spiral,

```
clear

R= [-0.6468    0.5740   -0.5022   %Rotation matrix
    -0.5073   -0.8155   -0.2786
    -0.5694    0.0745    0.8187];

theta=0:10:3*360;

xyzInput=R*[cosd(theta);sind(theta);theta/360]; %Helical input samples

fobj=cylindricalFit(xyzInput); %Perform the fit

[hFit,hData]=plot(fobj,{},{'MarkerFaceColor','g','MarkerEdgeColor','b','S

xlim([-2.38 0.87]); ylim([-1.84 1.07]); zlim([-0.47 3.10]); view([18.90 1
```

and then extract samples length-wise along the cylinder,

```
xyzPost=fobj.sample(0:60:360, linspace(0,fobj.height,20)); %Post-samples

hold on
hPost=scatter3(xyzPost{:},'filled','k','SizeData',50,'MarkerFaceAlpha',0.
hold off

legend([hFit,hData,hPost],'Cylinder fit','XYZ input samples','XYZ post-sa
                          'Location','southoutside','FontSize',15);
```





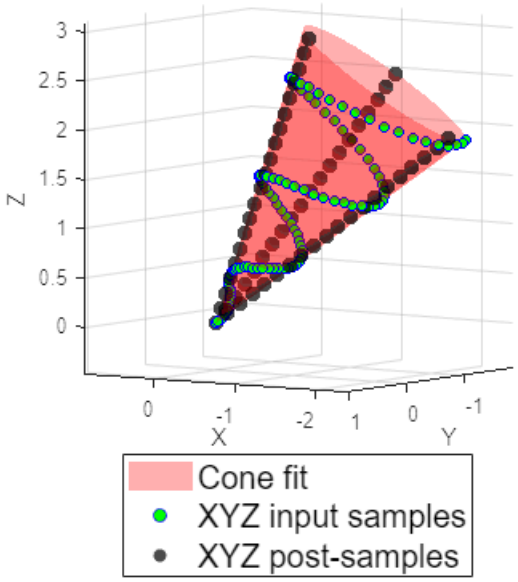
Post-Sampling a Cone Fit

Here, we fit spiraling samples on the surface of a cone,

```
clear

R= [-0.6468    0.5740   -0.5022   %Rotation matrix
    -0.5073   -0.8155   -0.2786
    -0.5694    0.0745    0.8187];

theta=0:10:3*360;

r=linspace(0,1, numel(theta));

xyzInput=R*[r.*cosd(theta);r.*sind(theta);theta/360]; %Helical input samp

fobj=rightcircularconeFit(xyzInput); %Perform the fit

[hFit,hData]=plot(fobj,{},{'MarkerFaceColor','g','MarkerEdgeColor','b','S

xlim([-2.38 0.87]); ylim([-1.84 1.07]); zlim([-0.47 3.10]); view([215.52
```

and then extract samples length-wise along the cone,

```
xyzPost=fobj.sample(0:120:360, linspace(0,fobj.height,20)); %Post-samples

hold on
hPost=scatter3(xyzPost{:},'filled','k','SizeData',50,'MarkerFaceAlpha',0.
hold off

legend([hFit,hData,hPost],'Cone fit','XYZ input samples','XYZ post-sample
                          'Location','southoutside','FontSize',15);
```



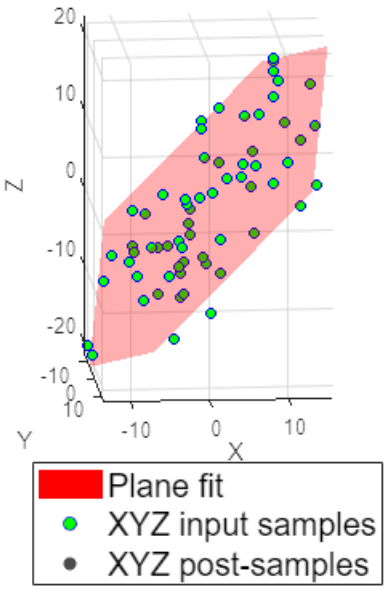
Post-Sampling a Plane Fit

In this example, after fitting a plane,

```
clear

b0=[1,0,0];  t=linspace(-10,10,8);

xyzInput=planarFit.xyzsim(b0,[0;1;1],[1;0;1],t,t,5); %Input samples

fobj=planarFit(xyzInput); %Perform the fit

[hFit,hData]=plot(fobj,{'MeshDensity',40},...
                       {'MarkerFaceColor','g','MarkerEdgeColor','b','Size

view([-47.9 3.6])
```

we show how to resample it along lines parallel and perpendicular to the x-z plane.

```
b1=cross([0,1,0],fobj.normal); %Make one sampling direction parallel to x
b2=[];                         %Make the other direction orthogonal to b1

xyzPost=fobj.sample(b0,b1,b2,2*t,t); %Post-samples

hold on
hPost=scatter3(xyzPost{:},'filled','k','SizeData',50,'MarkerFaceAlpha',0.
hold off

legend([hFit,hData,hPost],'Plane fit','XYZ input samples','XYZ post-sampl
                          'Location','southoutside','FontSize',15);
```



Ground Truth Objects

All the objects have a `groundtruth()` static method allowing one to generate a conic or quad
This is useful for viewing what a perfect fit would look like.

Ellipse Fit versus Ground Truth

Similar to earlier examples, we first generate simulated noisy samples of an ellipse,

```
clear

[x0,y0]=deal(5,2);  %Center coordinates
[a,b]=deal(15,7);  %Major and minor axis radii
angle=20;           %Rotation angle in degrees
```

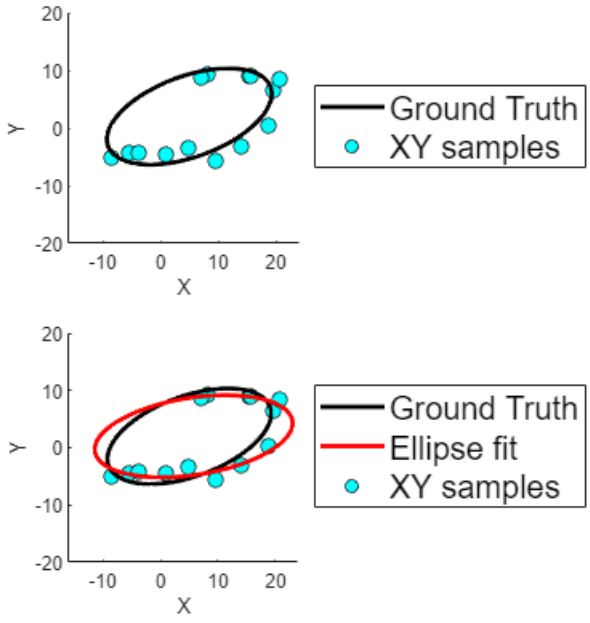
```
thetaSamps=-180:20:90; %Sample angles in degrees
sig=2;                 %Error standard deviation

xy=ellipticalFit.xysim([x0,y0],[a,b],angle, thetaSamps, sig); %Generate n
```

Next, we generate a ground truth object and plot,

```
gtobj=ellipticalFit.groundtruth(xy, [x0,y0],[a,b],angle); %Ground truth o

subplot(2,1,1)
[hTruth,hData]=plot(gtobj,{'Color','k','LineWidth',2},...
                          {'MarkerFaceColor','c','MarkerEdgeColor','k'});
legend([hTruth,hData],'Ground Truth','XY samples',...
                      'Location','eastoutside','FontSize',15);
 axis([-16 24 -20 20])
```

Or, if desired, we can add an actual ellipse fit on top of the data and ground truth locus,

```
subplot(2,1,2)
[hTruth,hData]=plot(gtobj,{'Color','k','LineWidth',2},...
                          {'MarkerFaceColor','c','MarkerEdgeColor','k'});
hold on
hFit = showfit(ellipticalFit(xy), 'Color','r','LineWidth',2); %Visualize
hold off

legend([hTruth,hFit,hData],'Ground Truth','Ellipse fit','XY samples',...
                    'Location','eastoutside','FontSize',15);

axis([-16 24 -20 20])
```

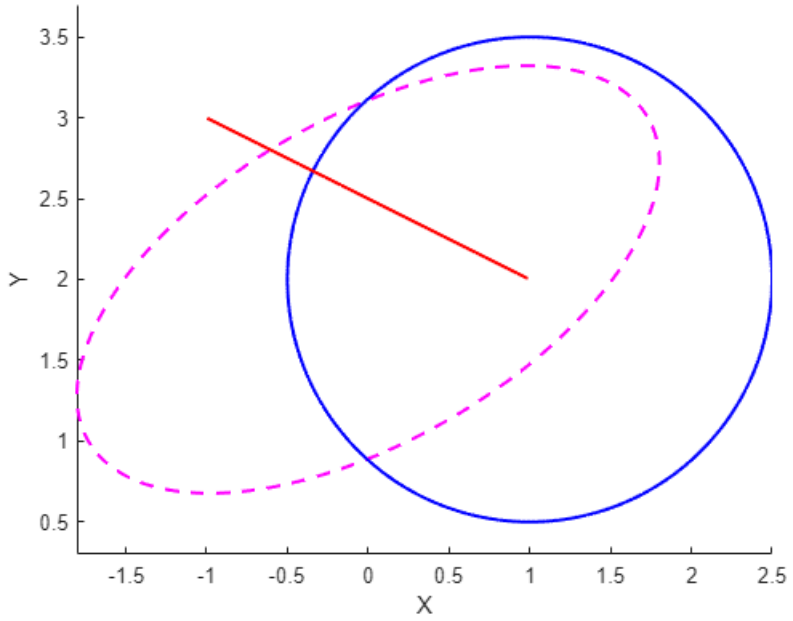


Plotting with "No-Data" Objects

A ground truth object does not actually have to be given any coordinate samples. Ground trut can be useful simply for plotting conics,

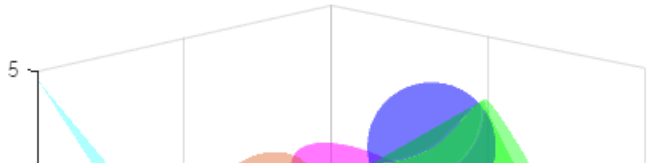
```
clear

[eCenter,eAB,eAngle]=deal([0 2] , [2 1], 30);    %Ellipse parameters
[cCenter,cRadius]=deal([1 2], 1.5);              %Circle parameters
        [p1,p2]=deal([-1 3], [1 2]);             %Line tips


gtEllip=ellipticalFit.groundtruth([],eCenter,eAB,eAngle);     %Data-less
gtCirc=circularFit.groundtruth([], cCenter, cRadius);
gtLine=linear2dFit.groundtruth([], p1, p2);
```

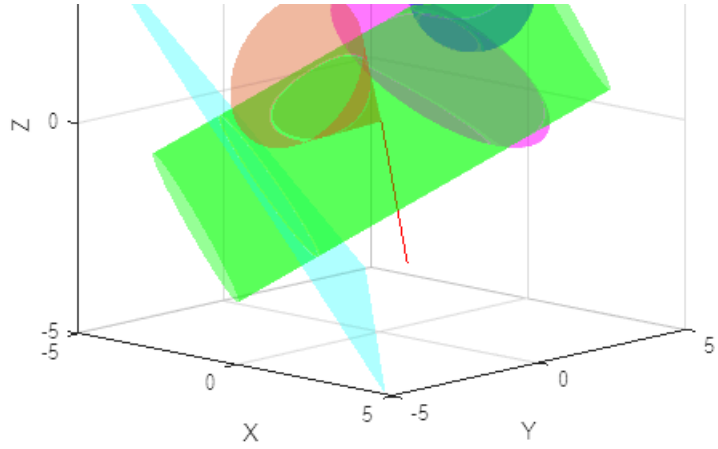
```
figure
hold on
 plot(gtEllip,{'Color','m','LineStyle','--'});
 plot(gtCirc,{'Color', 'b'});
 plot(gtLine,{'Color', 'r'})
hold off
```



and likewse 3D quadrics as well,

```
clear

[cCenter,cRadius,cHeight,cYP,cAngle]=...
                          deal([0 0 0], 2, 10, [45 -30],30);  %Cylinder/
[pNormal,pDistance]=deal([1 1 1], -3);                        %Plane par
[eCenter,eABC,eYPR]=deal([0 2 1] , [3 2 1], [20 30 0]);       %Ellipse p
[sCenter,sRadius]=deal([1 2 3], 1.5);                         %Sphere pa
          [p1,p2]=deal([0 0 0], [0 1 -4]);                    %Line tips


gtCyl=cylindricalFit.groundtruth([],cCenter,cRadius,cHeight,cYP); %Data-l
gtCone=rightcircularconeFit.groundtruth([],cCenter,cAngle,cHeight/3,cYP.*
gtPlane=planarFit.groundtruth([],pNormal,pDistance);
gtEllip=ellipsoidalFit.groundtruth([],eCenter,eABC,eYPR);
gtSphere=sphericalFit.groundtruth([], sCenter,sRadius);
gtLine=linear3dFit.groundtruth([], p1, p2);


figure
 hold on
 plot(gtCyl,     {'FaceColor', 'g','FaceAlpha',0.4});
 plot(gtCone,    {'FaceColor', [0.85, 0.33, 0.10],'FaceAlpha',0.4});
 plot(gtPlane,   {'FaceColor', 'c'});
 plot(gtEllip,   {'FaceColor', 'm'});
 plot(gtSphere,  {'FaceColor', 'b'});
 plot(gtLine,    {'Color', 'r'});
 view([43.03 12.32])
hold off
```





Advanced Topics

This section is devoted to some additional topics, which make more advanced use of the tool

Fitting a 2D Shape to 3D Points

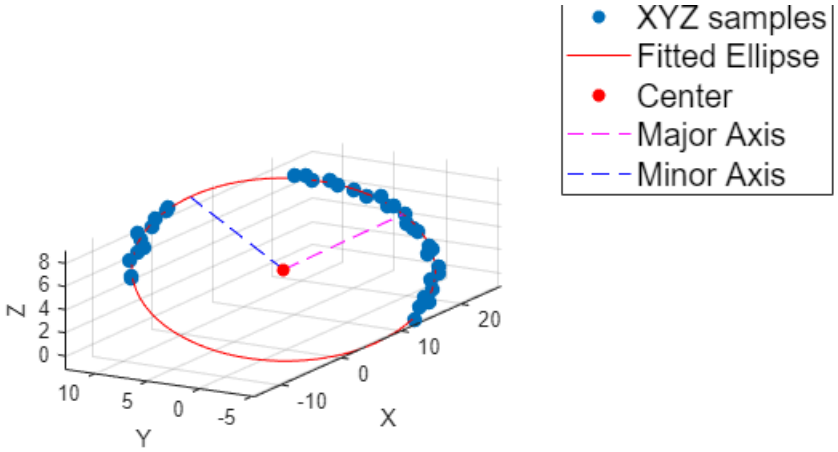
In some cases, one might wish to fit a 2D shape which is sampled in 3D space. The approach samples, then project the samples into a 2D coordinate system in this plane. Once projected previous sections can be applied freely.

This approach is illustrated below for the case of an ellipse fit from 3D sample points. Note ho `unproject3D` are used to facilitate the necessary coordinate transformations to 2D and back.

```
clear

[x0,y0]=deal(5,2);  %Center coordinates
[a,b]=deal(20,10);  %Major and minor axis radii
thetaSamps=[-70:5:50,100:5:150]; %2D sample angles in degrees
R=makehgtform('xrotate',pi/6); %3D rotation
t=[1;2;3];                     %3D translation

XYZ0=ellipticalFit.xysim([x0,y0],[a,b], 0, thetaSamps); %Generate noisy m
 XYZ0(3,:)=0;
 XYZ0=R(1:3,1:3)*XYZ0+t + 0.3*randn(size(XYZ0));

pFit=planarFit(XYZ0);%Preliminary plane fit

  xy0=pFit.project2D(XYZ0); %Map measured 3D samples to 2D

eFit=ellipticalFit(xy0); %Perform ellipse fit in 2D

  XYZ=pFit.unproject3D(  cell2mat(eFit.sample(1:360))  ); %Post-sample th
```

Here, we have used the `unproject3D()` method to map 2D samples, generated with the `eFi` coordinate system. However, we can likewise use it to find other landmarks of interest, such a ellipse:

```
Center=pFit.unproject3D(eFit.center(:));
Vertex  =pFit.unproject3D( eFit.center(:)) + eFit.a*pFit.unproject3D( eFi
CoVertex=pFit.unproject3D( eFit.center(:)) + eFit.b*pFit.unproject3D( eFi

scatter(pFit); hold on
plot3(XYZ(1,:), XYZ(2,:), XYZ(3,:),'r-');
scatter3(Center(1)  , Center(2), Center(3),'filled','r');
plot3([Center(1), Vertex(1)]  , [Center(2), Vertex(2)], [Center(3), Verte
plot3([Center(1), CoVertex(1)]  , [Center(2), CoVertex(2)], [Center(3), C
axis equal; view(-60,15); legend('XYZ samples','Fitted Ellipse','Center',
```



Version	Published	Release Notes	
1.2.11	8 Feb 2024	Small bug fix in ellipticalFit.xysim() and ellipticalFIt.sample()	Download
1.2.10	6 Feb 2024	Further description updates	Download
1.2.9	6 Feb 2024	Small modifications to description page.	Download

Version	Published	Release Notes	
1.2.8	25 Oct 2022	*Added some functionality to planarFit.project2D and planarFIt.unproject3D(). *Embellished example in "Fitting a 2D Shape to 3D Points", making it a 3D ellipse fit.	Download
1.2.7	25 Oct 2022	Minor edits to Examples.mlx	Download
1.2.6	25 Oct 2022	Added some tools and examples to help with fitting circles, ellipses, and other 2D shapes that are sampled in 3D. See the Examples.mlx section "Fitting a 2D Shape to 3D Points".	Download
1.2.5	24 Jun 2021	Bug fix in circularFit.sample().	Download
1.2.4	18 Jun 2021	Cosmetic update: default axis labels will now be provided on all plots.	Download
1.2.3	13 Jun 2021	Upload redo.	Download
1.2.2	13 Jun 2021	Updated Examples.mlx with examples for cones.	Download
1.2.1	12 Jun 2021	*Added rightcircularconeFit (beta version) *Added showfit() method as a general interface for plotting the fit only (without the data points) *Some documentation typos fixed. *Other cosmetic fixes.	Download
1.1.1	8 Jun 2021	Cosmetic fix: quadricFit.plot() was unintentionally returning an output argument when nargout=0.	Download
1.1.0	15 Mar 2021	*Added groundtruth() methods. *3D line plots now return line handles. *3D ellipsoid and cylinder plots now return surf handles (implicit surface behavior was problematic).	Download
1.0.5	19 Feb 2021	Small text corrections	Download
1.0.4	19 Feb 2021	Mild tweaks and fixes	Download
1.0.3	19 Feb 2021	Small edits	Download
1.0.2	19 Feb 2021	Small edit	Download
1.0.1	19 Feb 2021	Small edit to Examples.mlx	Download
1.0.0	19 Feb 2021		Download

Join the conversation